

Chapitre 3 : Cassandra

Kaouther Gasmi, PhD
kaouther.gasmi@fsb.ucar.tn

Présentation

- Conçu à l'origine par Facebook pour sa fonctionnalité de messagerie
- Ecrit en Java
- Actuellement projet de la fondation Apache (2008)
- Distribué par la société Datastax (en fournissant documentation environnement de développement et conseils)
- Initialement basé sur le système BigTable de Google (stockage orienté colonnes)
- Maintenant basé sur le système DynamoDB de Amazon
- Un des rares systèmes NoSQL à proposer un typage fort
-
- Un langage de définition de schéma et de requêtes, CQL (pas de jointure)
- Publiée en open source sur Google Code en juillet 2008

Le modèle de données

■ Colonne:

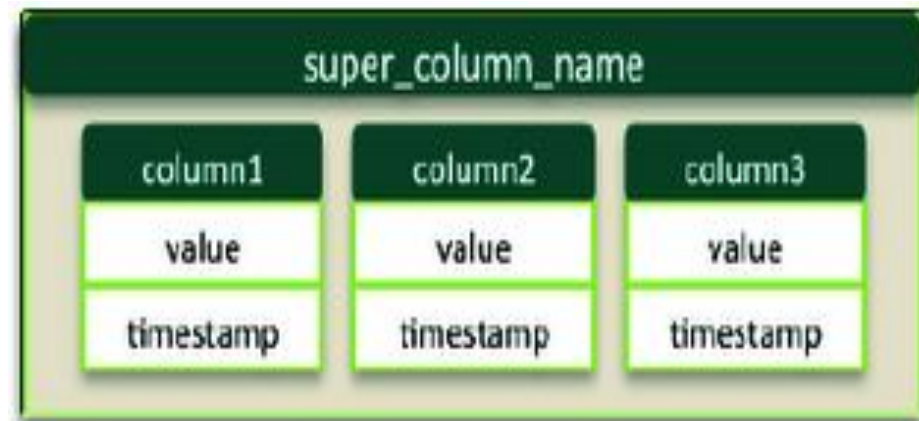
- La plus petite valeur possible dans un système Cassandra.
- C'est un triplet composé du nom de la colonne, de la valeur qu'elle possède, et de son timestamp. Ce dernier indique l'heure et la date à laquelle l'information a été enregistrée. Il est utilisé par le système pour savoir quelle est l'information la plus actuelle.
- Une colonne peut contenir d'autres colonnes nommée super-colonne

Nom
Valeur
<i>Timestamp</i>

Le modèle de données

■ Super-olonnes:

- Elle représente une colonne dont la valeur pouvant contenir d'autres colonnes.
- C'est un triplet composé du nom de la colonne, de la valeur qu'elle possède, et de son timestamp. Ce dernier indique l'heure et la date à laquelle l'information a été enregistrée. Il est utilisé par le système pour savoir quelle est l'information la plus actu



Le modèle de données

■ Ligne:

- Une ligne est un enregistrement. Elle est composée d'un ensemble de colonnes. Chaque ligne est identifiée par une clé, qu'on appellera « rowKey ».
- Il est possible d'utiliser des colonnes comme clé primaire d'une ligne.
- Une ligne peut contenir jusqu'à deux milliards de colonnes.

The diagram shows a table structure with two rows. The first row is identified by the key 'baronm' and contains columns: familyName (BARON), firstName (Mickael), age (36), and address (Poitiers). The second row is identified by the key 'duponto' and contains columns: familyName (DUPONT), firstName (Olivier), and phone (+335432312). Labels with arrows point to various parts: 'Ligne' points to the first row, 'Clé' points to the 'baronm' key, 'Colonne' points to the 'address' column header, 'Nom colonne' points to the 'phone' column header, and 'Valeur' points to the phone number value.

baronm	familyName	firstName	age	address
	BARON	Mickael	36	Poitiers
duponto	familyName	firstName	phone	
	DUPONT	Olivier	+335432312	

Le modèle de données

■ Famille de colonnes:

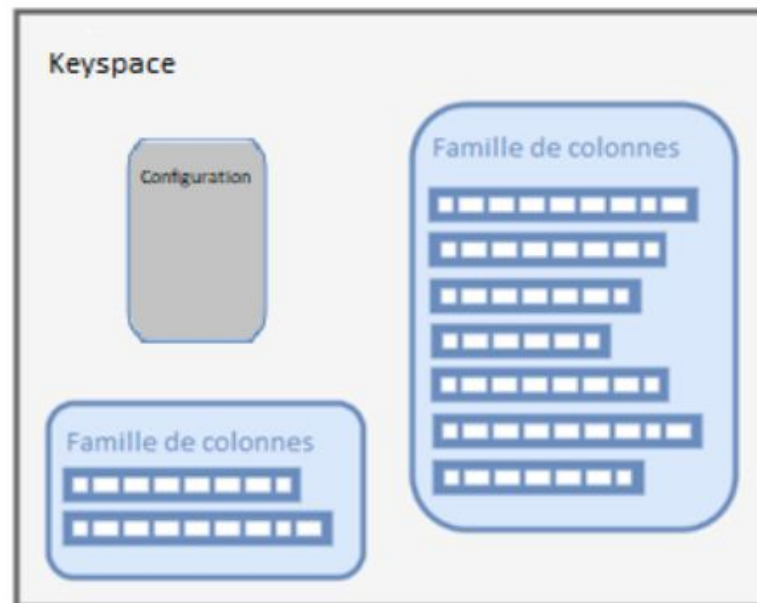
- Une famille de colonnes est un regroupement logique de lignes (d'enregistrements). Dans le monde relationnel, elle est comparable à une table.
- On distingue deux types de familles de colonnes :
 - **Famille de colonnes statique** : les colonnes sont définies lors de la création ou la modification de celle-ci,
 - **Famille de colonnes dynamique**: les colonnes sont définies lors de la création ou la modification d'une ligne.

Persons				
baronm	familyName	firstName	age	address
	BARON	Mickael	36	Poitiers
duponto	familyName	firstName	phone	
	DUPONT	Olivier	+335432312	

Le modèle de données

- Keyspace:

- Le keyspace est un regroupement des différentes familles de colonnes. On peut le considérer comme un schéma de base de données.



Le modèle de données

- Types d attributs:
 - Tout comme SQL, Cassandra propose plusieurs types d' attributs.

Type Interne	Nom CQL
BytesType	blob
AsciiType	ascii
UTF8Type	text, varchar
IntegerType	varint
Int32Type	int
LongType	bigint
UUIDType	uuid
TimeUUIDType	timeuuid
DateType	timestamp
BooleanType	boolean
FloatType	float
DoubleType	double
DecimalType	decimal
CounterColumnType	counter

Indexation et clés

Pour les SGBDR : on utilise les clés primaires, elles permettent d'accélérer l'accès aux données, garantie l'unicité d'un enregistrement et ordonner les données

- **Pour les bases NoSQL**, un moyen d'accéder rapidement à une donnée est essentiel étant donné le volume de données stockées.

Pour Cassandra, c'est particulier. Une famille de colonnes dispose de 2 index :

- **un index primaire** : c'est l'index des clés de ses lignes.
- **un index secondaire** : c'est un index portant sur une colonne.

Permet d'appliquer des prédicat d'égalité sur cette colonne (ex : ... WHERE colonne X = y)

Indexation et clés

Les clés

- Pour Cassandra, la clé de ligne peut être **composée d'une ou plusieurs colonnes**, la clé est alors composée. Dans le cas d'une clé composée, le **premier élément** est la **clé de partitionnement**, les suivants sont les **clés de cluster(isation)**.

Clé de partitionnement

- Elle détermine sur quel nœud du cluster la donnée sera stockée.

Clé de cluster(isation)

- Elle sert à ordonner les données sur un même nœud.

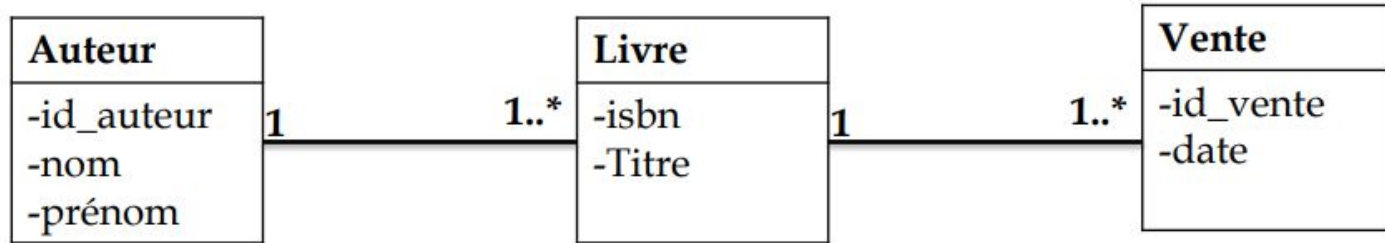
Indexation et clés

- Exemple : Soit la table ayant le schéma suivant :

```
CREATE TABLE message (  
  user_id bigint, date int, message_id timeuuid, content  
  text, status text, PRIMARY KEY (user_id, message_id)  
)
```

- tous les messages de même utilisateur sont stockés sur le même noeud
- `SELECT * FROM message WHERE id = 46196ee77204 ORDER BY message_id DESC LIMIT 50;`
- La clé de clusterisation nous permet d'utiliser ORDER BY sur ses données une fois que l'on sait sur quel nœud se situe les données.

Exercice : Du relationnel aux modèles NoSQL



- On considère le cas d'usage suivant: Accéder aux livres d'un auteur donné (recherche à partir d'un nom donné).
- Proposer une modélisation orientée colonne pour ce cas d'usage

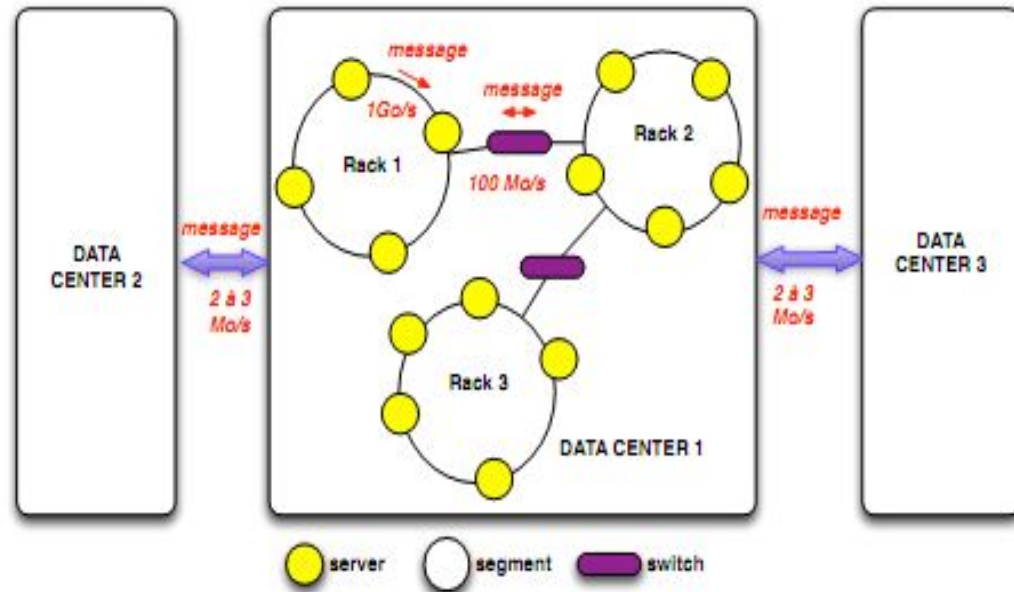
Cluster Cassandra

- **Scalabilité horizontale (scale out)** : Pouvoir augmenter le nombre de machines autant que nécessaire pour traiter de plus gros volumes de données.
- **Scalabilité verticale (Scale up)**: possibilité d'ajouter des ressources supplémentaire à une machine (CPU, RAM, disque...).
Coûte plus cher et rencontre forcément une limite
- Dans une BD Nosql la stratégie de distribution des données sur un ensemble de machines (scale out) a pour premier but un passage à l'échelle . C'est à dire de permettre d'écrire et de lire de plus gros volumes de données.
- Leurs mécanismes de distribution des données visent plutôt à équilibrer la charge des noeuds de données

Cluster Cassandra

Architecture générale d'un cluster :

- **Un nœud** (node) : C'est un serveur physique qui contiendra les différentes données de la base. Les nœuds sont regroupés en « Rack : liaison réseau rapide, environ 1Go/sec
- **Un centre de données** (Data-Center) : consiste en un nombre de « racks » : liaison à Mo/sec
- entre différents « Data centers » 2-3 Mo/sec
- Les nœuds communiquent par envoi de messages.



Cluster Cassandra

- Ex. : Data center de Google (début 2010) :
- Un « Data center » Google contient entre 100 et 200 « Racks », chacun contenant 40 serveurs
- environ 5000 serveurs par « Data-center » pour un total de 1 millions de serveurs (estimation d'après la consommation électrique) ;
- Estimation Garner de 2,5 millions de servers pour Google en 2016 ...

Cluster Cassandra

- Un Cluster Cassandra fonctionne en mode **multi-maître**. La notion de nœud maître et nœud esclave n'existe donc pas.
- Chaque nœud du cluster a le même rôle et la même importance, et jouit donc de la capacité de lecture et d'écriture dans le cluster.
- Un nœud ne sera donc jamais préféré à un autre pour être interrogé par le client.
- Chaque nœud sait donc où sont les autres nœuds, quels sont leurs identifiants, quels nœuds sont disponibles et lesquels ne le sont pas.
- Lorsque qu'un client veut lire ou écrire une donnée, il lui suffit de se connecter à l'un des nœuds du cluster, si le nœud en question ne contient pas la donnée, alors il la demandera au nœud qui la contient. Ce mécanisme est transparent pour le client

Cluster Cassandra

Communication inter-nœuds

- Cassandra utilise un protocole appelé "**gossip**" pour découvrir des informations sur l'emplacement et l'état des autres nœuds participant à un cluster Cassandra.
- Le Gossip est un protocole de communication pair-à-pair dans lequel les nœuds échangent périodiquement des informations sur leur état et sur celui des autres nœuds qu'ils connaissent.
- Dans Cassandra, le processus Gossip s'exécute toutes les secondes et échange des messages d'état avec un maximum de trois autres nœuds dans le cluster.
- Ainsi, tous les nœuds se renseignent rapidement sur tous les autres nœuds du cluster

Cluster Cassandra

- Un cluster peut contenir plusieurs Keyspaces.
- Un Keyspace est l'équivalent d'un schéma dans une BDD relationnelle.
- Chaque Keyspace peut être paramétré de manière indépendante.
- Les paramètres rentrant en jeu sont le Replication Factor (RF) et le Consistency Level (CL).
- Ces paramètres permettent de gérer la disponibilité (A) et la résistance au morcellement (P) de Cassandra.

Distribution de données

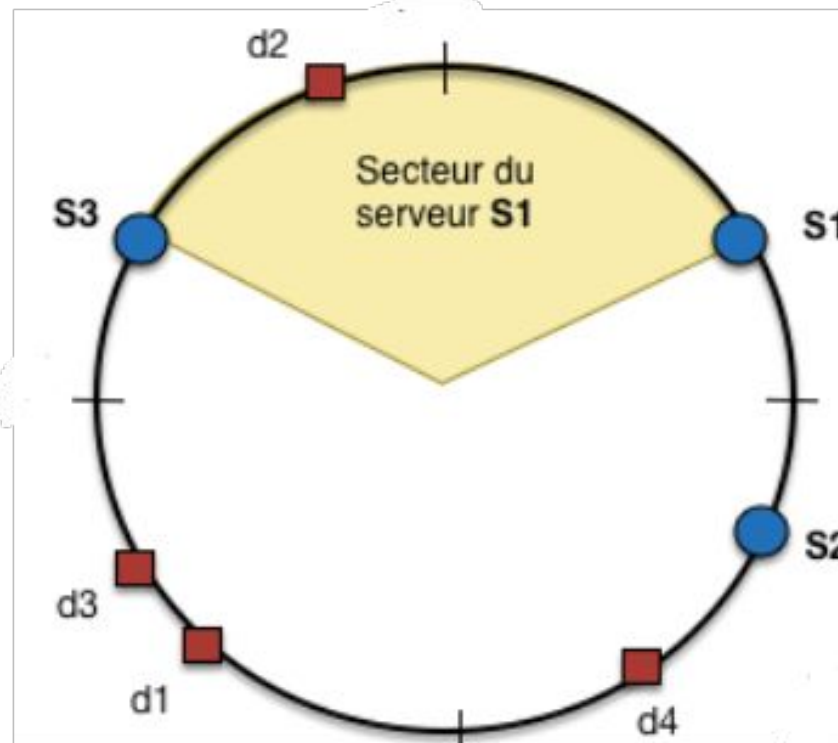
■ Consistent hashing:

- Mécanisme de partitionnement dans lequel les lignes sont stockés sur des nœuds-serveurs différents en utilisant une **fonction de hachage** (par défaut, un algorithme nommé MurMur 3, plusieurs autres choix sont possibles: md5, lexicographique.)
- Le principe du hachage cohérent est de considérer dès le départ un intervalle immuable $D = [0, n-1]$ pour le domaine d'arrivée de la fonction de hachage, où n est choisi assez grand pour réduire le nombre de collisions
- Les nœuds sont affectés par la fonction de hachage à un anneau (rack/ cluster de serveurs , ou *hash ring*
- Un serveur **S** obtient un token t_n
- Le serveur **S** devient alors responsable de l'intervalle de valeurs de hachage $] t_{n-1} - t_n]$
- Au moment d'une insertion, la **même fonction de hachage** est appliquée à la **clé primaire** de la ligne , et le résultat détermine le serveur sur lequel la ligne est insérée

Distribution de données

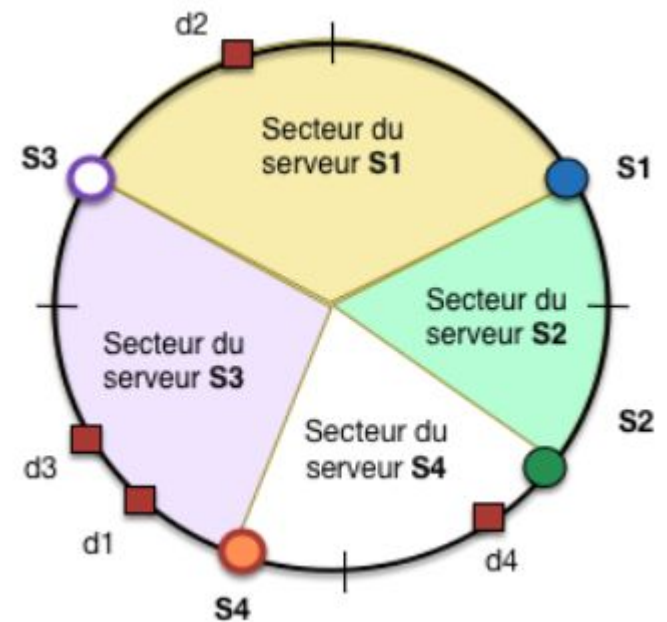
- Consistent hashing:

- Exemple



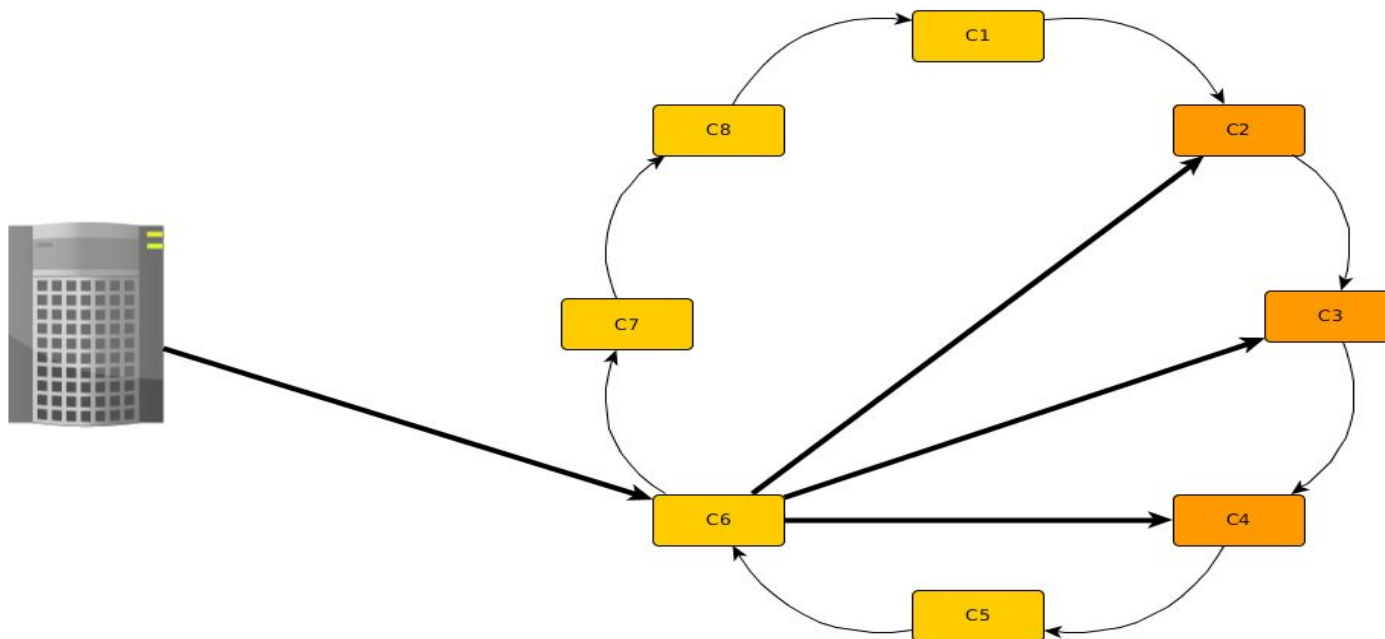
Distribution de données

- **Ajout/suppression de serveurs:**
 - Quand un nouveau serveur est ajouté, ses nouvelles positions sont calculées, et chaque insertion à une position implique une division d'un arc de cercle existant.
 - quand un nœud quitte l'anneau, les lignes qui lui sont liés sont alors associés à leur **nœud adjacent dans le sens horaire**



Routage de requêtes d'écriture

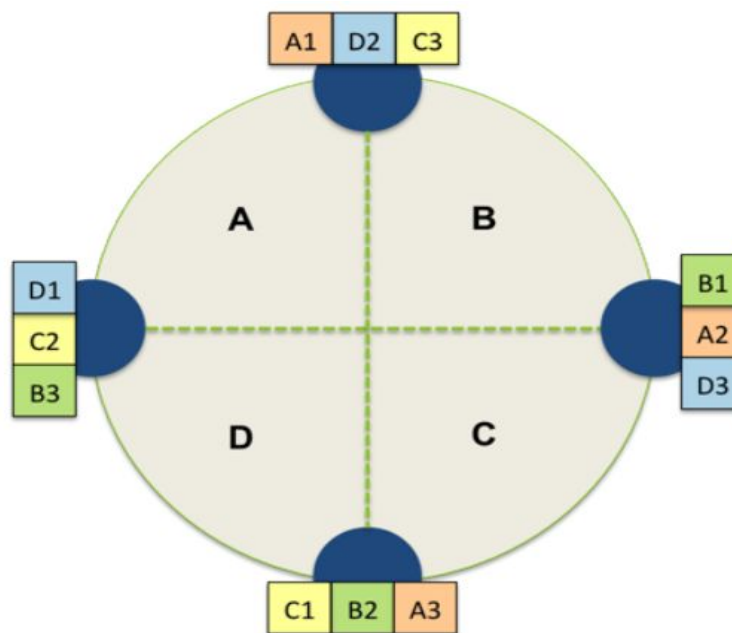
- Un client qui interroge Cassandra contacte un nœud coordinateur au hasard parmi tous les nœuds du cluster.
- Le rôle du coordinateur : au lieu de se charger lui-même d'une écriture locale, puis de transmettre des demandes de réplication, le coordinateur envoie f demandes d'écriture en parallèle à f nœuds de l'anneau, où f est le facteur de réplication



Stratégies de réplication

Il existe 2 types de stratégies :

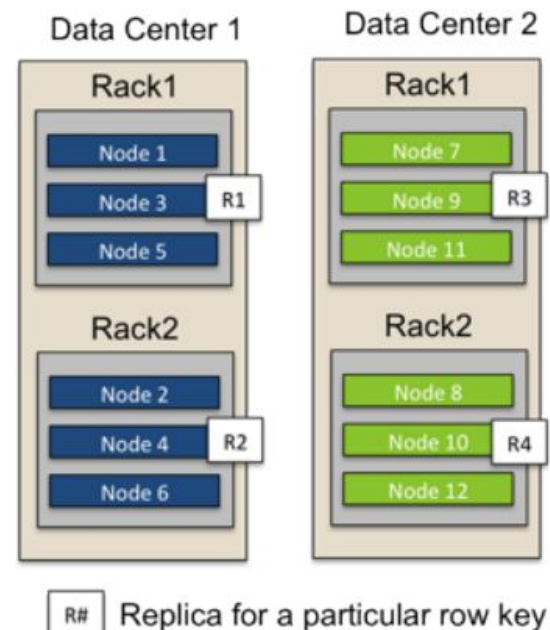
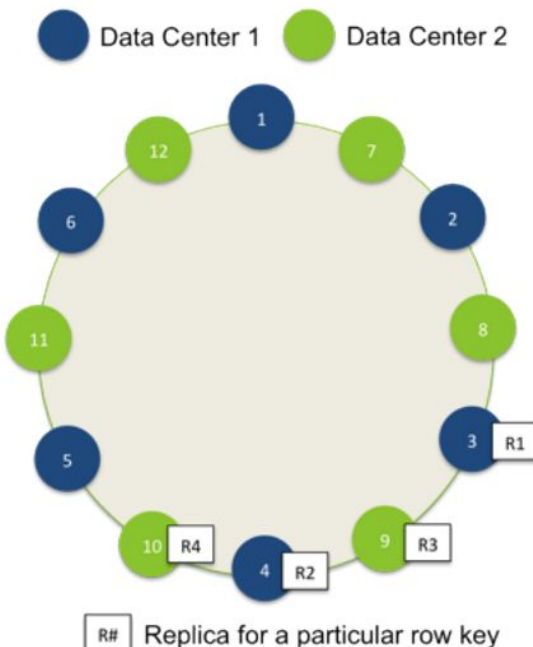
- **SimpleStrategy** : . La première copie est placée dans le noeud déterminé à partir de la clé de partition, les suivants sont placés sur les noeuds voisins dans le sens des aiguilles d'une montre sans se soucier de la topologie (localisation du data-center ou de la rack).



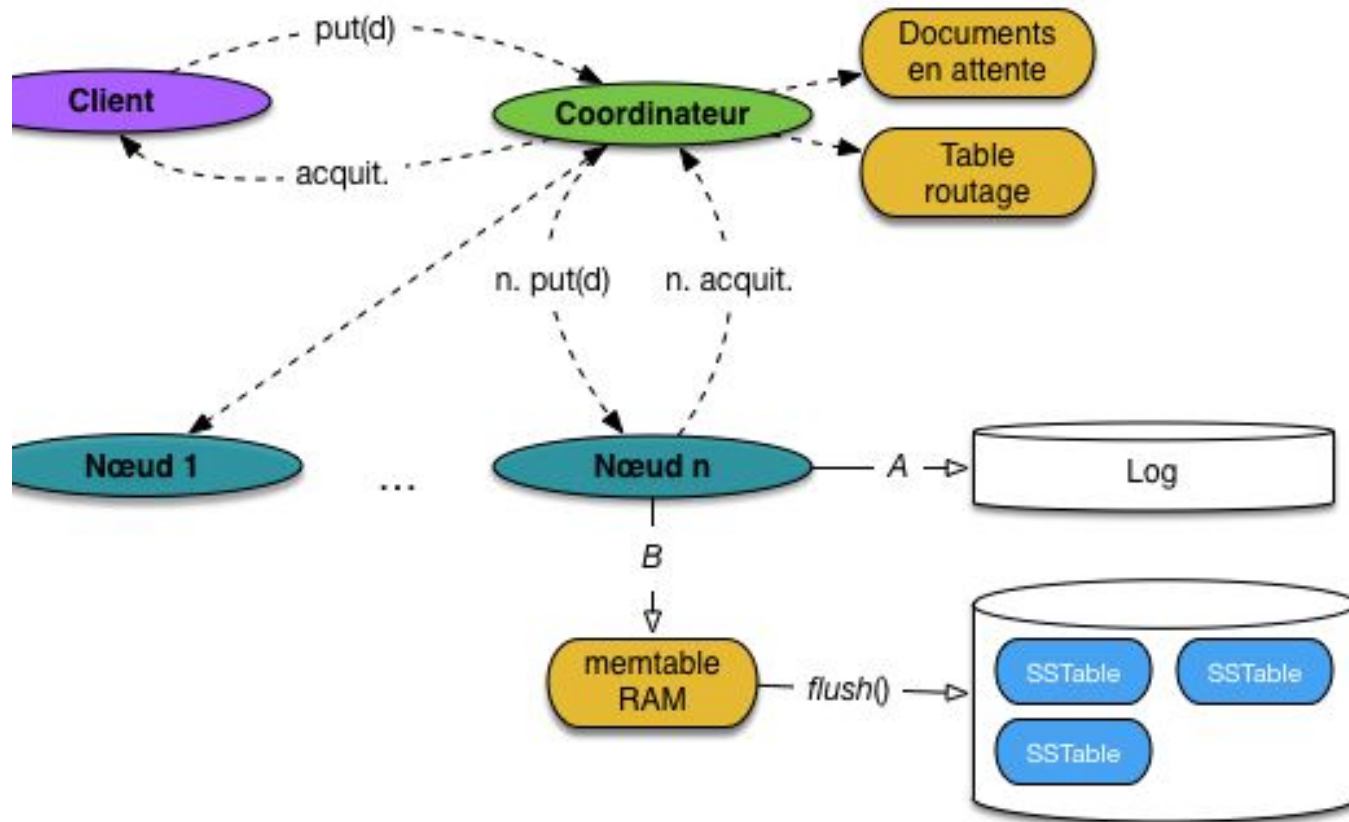
Stratégies de réplication

Il existe 2 types de stratégies :

- **NetworkTopologyStrategy** : à utiliser lorsque le cluster est déployé sur plusieurs data-centers. Cette stratégie permet de spécifier le nombre de copies voulu dans chaque data-center, elle permet également de placer les copies sur des racks distincts car les noeuds d'une même rack rencontre souvent simultanément des problèmes d'alimentation, de refroidissement ou de réseau.



Écriture et cohérence de données



Écriture et cohérence de données

- Une application cliente a donc pris contact avec un des serveurs, que nous appelons le *coordonateur*.
- Ce dernier reçoit une demande d'écriture d'une ligne d . Cette ligne est immédiatement écrite dans le fichier *log* (flèche A) et placé dans une structure en mémoire, appelée *memtable* (flèche B).
- À ce stade, on peut considérer que l'écriture est effectuée de manière durable
- un *flush()* est effectué pour transférer son contenu sur disque, dans un fichier structuré sous forme de **SSTable**
- Si l'un des nœuds sur lesquels on doit écrire une ligne soit indisponible, le coordonateur peut attendre que le nœud soit de nouveau actif avant d'écrire. Bien entendu, rien ne l'empêche d'écrire sur les autres réplicas.
- Lorsque le coordonateur attend la remise à disponibilité du nœud, il stocke temporairement la donnée localement.
 - la donnée n'est pas à proprement parler dans le cluster pendant cette attente. Il n'est donc pas disponible à la lecture. Ce cas de figure (extrême) s'appelle un **Hinted Handoff**.

Écriture et cohérence de données

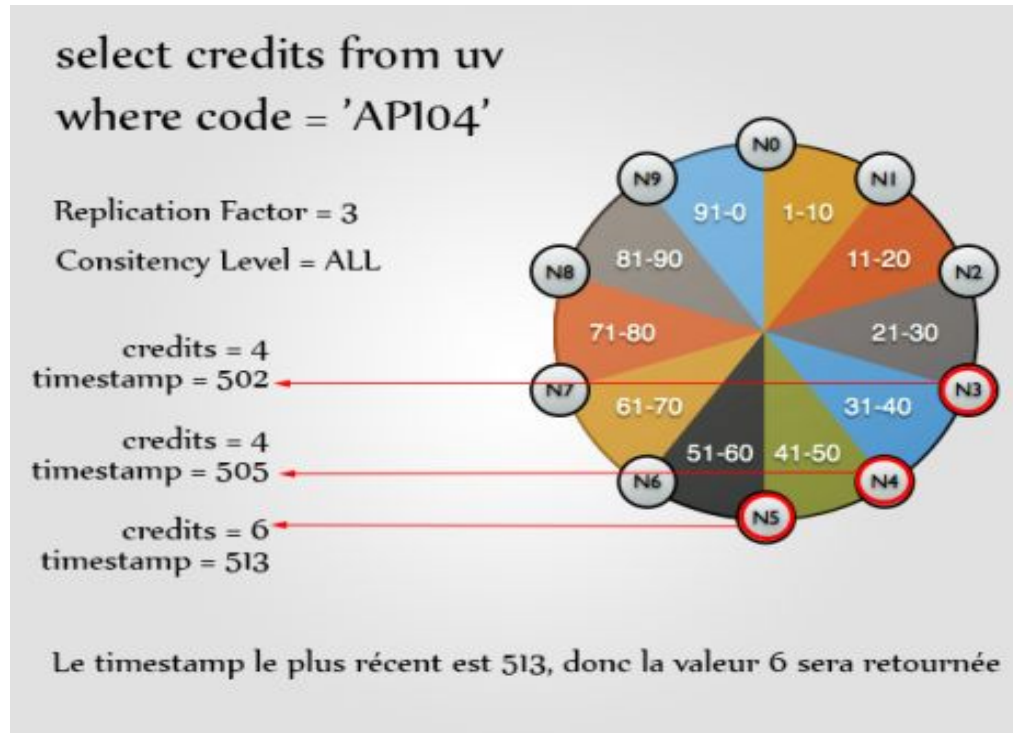
- Cassandra dispose de paramètres de configuration avancés qui servent à ajuster le compromis entre latence, cohérence, disponibilité et la tolérance aux pannes
- **Niveau de cohérence des écritures** : combien de répliques doivent être écrites avec succès avant retourner un acquittement au client.
- Il existe plusieurs valeurs possibles pour le niveau de cohérence, dont les plus importantes :
 - **ONE** : l'écriture est validée après écriture sur un seul nœud.
Satisfait les besoins de la plupart des utilisateurs car les exigences de cohérence ne sont pas strictes.
 - **QUORUM** : l'écriture est validée après écriture sur $(RF/2) + 1$ nœuds.
À utiliser si vous pouvez tolérer un certain niveau d'échec.
 - **ALL** : l'écriture est validée après écriture sur RF nœuds
Fournit la plus grande cohérence et la plus faible disponibilité de tous les autres niveaux.
 - **ANY** : Une écriture doit être effectuée sur au moins un nœud. Si tous les nœuds de réplique sont hors service au moment de l'écriture, le *Hinted Handoff* est appliqué

Lecture et cohérence de données

La lecture avec Cassandra est plus coûteuse que l'écriture. Sur chaque nœud, il faut chercher la donnée dans la *memtable* (en RA et dans toutes les *SSTables*). Cela peut entraîner des accès disques, et donc une pénalité assez forte.

- **Niveau de cohérence de lecture:** détermine le nombre de réplicas doivent répondre avant que le résultat ne soit fourni au client.
- Cassandra contacte le nombre de réplicas indiqué afin d'obtenir la donnée la plus récente (en se basant sur le *timestamp*)
- Il existe plusieurs valeurs possibles pour le niveau de cohérence, dont les plus importantes :
 - **ONE** : obtention d'une réponse à partir du nœud réplique la plus proche.
 - **QUORUM** : obtention du résultat le plus récent à partir de $(RF/2) + 1$ nœuds.
 - **ALL** : Obtention du résultat le plus récent à partir de tous les nœuds répliques

Lecture et cohérence de données



lorsque client fait une lecture, tous les nœuds contenant la donnée doivent donc retourner une réponse. Cependant, les données sont ici différentes, dans ce cas on récupère la donnée qui a été ajoutée le plus récemment en utilisant le timestamp.

Les noeuds ayant 4 pour valeur de "credits" ne vont pas tarder à avoir la valeur 6. La mise à jour n'a pas encore été effective.

Impact sur les performances

Pour assurer la cohérence des lectures, il faut *toujours* que la dernière version d'une ressource fasse partie des réplikas obtenus avant réponse au client. Une formule simple permet de savoir si c'est le cas. Si on note

- W le nombre de réplikas requis en écriture,
- R le nombre de réplikas requis en lecture,
- RF le facteur de réplication. alors la cohérence est assurée si

$$R+W>RF$$

- si $W=ALL$ et $R=1$: on se satisfait d'une lecture, mais comme tous les écritures sont synchronisées, on est sûr qu'elle renvoie la dernière mise à jour.
- si $W=1$ et $R=ALL$, le raisonnement réciproque amène à la même conclusion
- enfin, si $W=QUORUM$ et $R=QUORUM$, on est sûr d'être cohérent: c'est la configuration la plus flexible, car s'adaptant automatiquement au niveau de réplication.

Impact sur les performances

La lecture pour le client: Pour les valeurs de CL :

- ONE : Une augmentation du RF améliore les performances en lecture du point de vue du client. Il y a plus de chance que le client demande la donnée à un nœud qui la contient et donc cette dernière serait retournée plus rapidement. GAIN
- QUORUM/ALL : Une augmentation du RF augmente le nombre de nœuds devant être consultés avant que la donnée ne soit retournée, ce qui augmente le temps de réponse. PERTE

L'écriture pour le client: Pour les valeurs de CL :

- ONE : Une augmentation du RF n'impacte pas le temps mis par le cluster à valider l'écriture du client. (le nœud sur lequel la donnée va être écrite dépend de la clé de partition et non du RF). IDEM
- QUORUM/ALL : Une augmentation du RF augmentera le nombre de nœuds devant valider l'écriture de la donnée avant que la validation d'écriture ne soit retournée au client, ce qui augmente le temps de réponse. PERTE